照着这条链路实现，再用事件回放验收

第一版目标：内容状态正确、任务可恢复、延时可解释。具体阈值与企业参数由你填写。

当前已有：模型Port、PG知识存储、Milvus适配、分层检索核心；待接：持久任务、有效版本校验、迁移恢复与统一查询。

先打通一个闭环，再扩成三层

① 企业内容接口
news_id / 修订号 / 正文
发布时间 / 撤稿 / 游标

→

② PostgreSQL
同事务：文档状态 + 索引任务
版本 / 有效投影 / generation

→

③ Temporal Worker
确定性切片 → Embedding
分批 / 限流 / 超时 / 重试

→

④ Milvus三集合
HOT / WARM / COLD
写入 → 校验完整与可见

→

⑤ 条件激活
PG核对版本和generation
生效后清理旧投影

查询入口　同版本Query向量 → 三层并发 → PG有效状态校验 → 新闻去重 / 融合 → 证据与耗时Trace

四种内容变化：先写清允许服务哪个版本，再写代码

新增
落库待处理 → 新向量可检索
→ PG激活 → 查询可返回

更正
PG先阻断旧版 → 构建新版
→ 条件激活；期间可暂缺该稿

撤稿
PG提交墓碑 → 新查询屏蔽
→ 异步清理全部层的旧切片

迁移 / 回热
复制 → 验证 → PG切有效层
→ 清理；旧任务不得复活内容

参数分三类填写

先冻结身份：模型revision / 维度 / 归一化 / metric / 切片版本
再定策略：年龄边界Th<Tc / 回热保持 / 迁移周期
最后调容量：候选切片数 / ef / batch / 并发 / deadline

ef约束看实际切片limit；chunk_chars是字符，不是token。

现有YAML不接受新字段；待实现参数在填写表中单独标明。

分层差异怎样测出来

固定业务时钟，让新闻跨边界；逐条回放更正、撤稿、回热。
耗时用真实单调时钟；记录三层数量、变更滞后、各层p95。
三集合共用资源不保证冷热延时差；稳定后再改变资源/索引。

并发全查：总耗时受最慢层影响，分层本身不保证更快。

注入sleep/网络延时用于故障与超时验证，不能当真实性能。

每一关有证据，才进入下一关

G1 合同与单篇
真实模型 + 隔离集合

G2 入库恢复
落库 / 派发 / 重启

G3 查询正确
版本 / 租户 / 去重

G4 生命周期
更新 / 撤稿 / 迁移

G5 故障回放
重复 / 乱序 / 断连

G6 混合负载
延时 / 新鲜度 / 积压

交付每轮参数快照 + 事件序列 + 查询Trace + 失败明细；通过阈值由你约定，逻辑错误、撤稿残留和跨租户返回必须为零。

本图为实施指导，不表示待开发能力已接入。参考：Milvus HNSW / Consistency；Temporal Retry Policies。